Priyansu Biswal

+1 (210) 396-6647 | pribiswal7@gmail.com | linkedin.com/in/priyansubiswal | github.com/Pri1s | priyansubiswal.dev

Education

Texas A&M University College Station, TX
B.S. in Computer Science (Engineering Honors) *Expected May 2028*
- **Relevant Coursework:** Program Design & Concepts, Data Structures & Algorithms, Discrete Structures & Computing

Technical Skills

Languages: Python, TypeScript, SQL, JavaScript, Java, C++, Verilog
Frameworks: FastAPI, Next.js, React.js, SQLAlchemy, Pydantic, Node.js, REST APIs, cocotb
AI/ML: LLM APIs (Claude, OpenRouter), Agentic AI, MCP, RAG, PyTorch, XGBoost, YOLO11, pandas, NumPy
Databases: PostgreSQL, pgvector, Redis
Cloud/Tools: Docker, Kubernetes, Azure (Container Apps, AKS, ACR), AWS, GitHub Actions, Pytest, Playwright, Git, Yosys

Experience

Team Lead – Future Market Prediction Sept. 2026 – Present
Aggie Data Science Club (American Airlines Partnership) *College Station, TX*
- Leading **4 students** with American Airlines' Network Planning team to forecast airline market demand up to 2 years out
- Benchmarked ticket-price models on **300K+ flights** with 5-fold cross-validation; a Random Forest cut average error **45%** vs. XGBoost (**7%** vs. **14%** average error)
- Audited the model for data leakage, finding row order (the CSV was sorted by seat class) ranked 2nd in feature importance, then confirmed with cross-validation that accuracy held (R^2 **0.978** → **0.977**)

Software Engineer Intern June 2026 – Aug. 2026
Finch *College Station, TX*
- Built backend and AI features for Finch, a **250+ user** job-search app that matches jobs and tailors resumes with LLMs
- Built a RAG pipeline that retrieves jobs from **93K** pgvector embeddings for LLM fit scoring, filtering by seniority so intern searches stop returning senior roles; cut vector search from **8.3 s to 1.3 ms** with an HNSW index
- Sped up the first pass of the user profile builder (which turns a user's LinkedIn, GitHub, and resume into a structured profile) **2.19**× by moving it to a faster LLM, with no drop in output quality in side-by-side tests
- Cut failed profile builds from **38% to 5%** with the team by catching cut-off LLM responses, forcing structured JSON output, and splitting each project into its own parallel LLM call
- Automated releases with GitHub Actions CI/CD that tests every PR (Pytest, Playwright) and deploys to staging and production, removing manual release steps for **all 5 developers**

Quant Engineer Jan. 2026 – Present
Maroon Fund at Texas A&M (Student-Managed, $70K AUM) *College Station, TX*
- Caught a train/test leak inflating the fund's backtest to **104%** (the holdout overlapped labeled training dates); rebuilt validation with causal features and a clean **263-day** holdout covered by **25** unit tests
- Built a day-by-day trade simulator with a 3-source data fallback (Alpha Vantage, yfinance, CSV); on the holdout, the strategy returned **65%** after costs vs. **26%** for holding NVDA (Sharpe ratio **1.43**)
- Added risk analytics (Sortino, max drawdown, alpha, beta) benchmarked against the S&P 500, showing the strategy captured **137%** of its up-day gains but only **89%** of its down-day losses

Projects

RTL Debug Agent | *Python, Verilog, cocotb, Yosys, MCP, Docker* Sept. 2026 – Present
- Built an agentic AI system that debugs Verilog chip designs with multiple hidden bugs, using LLM tool calling to write and run its own functional verification tests from a plain-English spec
- Raised the bug-fix rate to **96%** with a tool-calling agent that simulates and revises its fixes, vs. **65%** for single-pass generation and **82%** for cost-matched best-of-3 sampling with the same Claude model
- Showed that simulation drives the gain: removing the agent's test-running tool dropped fixes from **96% to 71%**
- Built an MCP tool server (read, edit, lint, simulate, submit) that runs AI-written code in a network-isolated Docker sandbox, plus a benchmark of **1,056** bugs, each mathematically proven to break the circuit

Search Engine | *Python, FastAPI, PostgreSQL, Redis, Docker, Kubernetes* Dec. 2025 – Oct. 2026
- Built a web search engine from scratch that crawls, indexes, and ranks pages with field-aware BM25, returning the right page as the top result for **90%** of test searches
- Replaced a global delay with a shared Redis frontier and per-domain limits for **5x** faster crawling (27 → 135 pages/min)
- Made saving each new page **100x faster** with a write-ahead log, so newly crawled pages become searchable **twice as fast**
- Deployed the crawler, indexing worker, and search API as independently scaling services on Azure Kubernetes Service with managed PostgreSQL and Redis, isolating failures so one crashed service can't take down the rest

Next Up | *Python, PyTorch, YOLO11, Ultralytics* May 2026 – Present
- Building a high school basketball film analytics tool, where no public models exist and courts overlap other sports' lines
- Fine-tuned YOLO11-pose on hand-labeled frames to detect **22 court landmarks**, raising mAP **3x** (16.6% → 51.2%)